Hierarchical Goal Induction

Remy Ochei

The Bracket Studio

do.infinitely@gmail.com

Abstract

We present *Hierarchical Goal Induction* (HGI), an architecture for training goal-conditioned agents that map directly from perception and goal state to action. The system comprises four interlocking components: (1) a hierarchical detection module that recursively segments both spatial observations and temporal action histories via a scale-and-pad operation; (2) a hierarchical goal inducer that detects plan boundaries in observation streams, bootstrapped through weak supervision from operating-system accessibility trees and cloud-hosted language model annotations, then distilled to local operation; (3) a goal-conditioned preference model that scores candidate actions given history and goal; and (4) an autoregressive action model conditioned on both history and a writable scratchpad tensor encoding the agent's current objective. An iterative distill–search–distill loop refines the actor across successive generations. The endpoint is a *goal-to-action mapper*: a network with a designated input region—the scratchpad—where a goal can be inscribed, and from which the agent emits actions rather than explanations. The scratchpad is the critical design surface: whoever writes it determines what the agent optimizes for.

Keywords: goal-to-action mapping, hierarchical detection, plan segmentation, scratchpad tensor, goal-conditioned preference model, iterative distillation, teacher forcing, accessibility tree, weak supervision

1 Hierarchical Detection

The general system works as follows. We have a "retina" that feeds into a detection model. In the case of vision, we output the top-left and bottom-right pixel coordinates on our retina for a detected object, along with a class label. In the case of audition, operating on a mel spectrogram, we output the bounds of our detection (either a bounding box or a temporal span) together with a frequency mask, so that we know which frequencies belong to our sound.

We scale up and pad our detection, then feed it into the hierarchical detector again

recursively until we produce the null detection. We also move "along" a level by conditioning on the previous detection.

2 The Hierarchical Goal Inducer

Now I will describe the hierarchical goal inducer. Its behavior is simple: conditioned on the contents of its "history retina," it outputs the start and end of any plans it sees being executed.

How do we train such a system? We can exploit the accessibility tree in macOS. As we construct the history on the retina (essentially a video), we simultaneously create a parallel JSON-based log of events. We send this log to our favorite LLM provider to detect plans and label them with descriptions. This is the level-0 training signal for our hierarchical goal inducer.

Now that we have a trained goal inducer (outputting only the temporal span of each plan for now), we can take new action trajectories and delimit goals at multiple levels of the hierarchy.

3 The Goal-Conditioned Preference Model

Next, we produce a model that, conditioned on the history we have on our retina and the current goal, takes in an action and outputs a probability. This is our goal-conditioned preference model.

4 The Action Model

We attach a GPT-style head conditioned on our history and the contents of a scratchpad, and predict a payload representing our next action at each timestep. This is our first action model.

The action model is autoregressive in two dimensions: along the payload dimension and along the temporal dimension. The generation of each new payload is conditioned on the previous one.

5 The Training Loop

We train the action model through teacher forcing, then sample from the trained action model to generate data for the preference model.

Once we have a trained preference model, we can use search of all kinds to maximize the probability of an action. This constitutes a second "action" model—here we apply brute force, guided by the preference model.

To recap:

1. Train a GPT-style model to predict actions via teacher forcing.
2. Sample this model to generate training data for the preference model.
3. Apply brute-force search to maximize the preference model's output.
4. Distill the findings via supervised learning on context–action pairs discovered through brute-force search. Here we must search for the optimal architecture; there is no way around it.

So we started with the GPT-style predictor, but we were able to level up to a leaner, meaner model. We can now replace the original GPT model with our distilled version and run the process again.

6 Hierarchical Plan Detector v2.0

The first version of the hierarchical plan detector only detected temporal bounds. Now we add a GPT-style head conditioned on our history and apply it recursively, cropping the history temporally and "scaling up" in the time dimension.

We train both the bounds detector and the GPT head together.

To recap:

1. **Stage 1:** Train a plan-bounds detector that we can use hierarchically with a scale-and-pad operation.
2. **Stage 2:** Add a GPT-2-style head that we teacher-force with the cloud LLM's outputs.
3. **Stage 3:** We no longer need the cloud LLM provider and can detect and annotate nested plans locally.
4. **Stage 4:** We fill our "actor" network's plan scratchpad with the outputs of the distilled plan inducer. The actor's action predictions are trained conditioned on the scratchpad's contents.

7 The Goal-to-Action Mapper

Finally, we have an agent with a trained "scratchpad"—a designated region in the network where we can inscribe a goal, and the agent will attempt to achieve it. It will not try to talk its way there; instead, it will output actions.

We have arrived at the goal-to-action mapper architecture. *QED.*

8 Related Work

The HGI architecture draws on and departs from several lines of prior work. The hierarchical decomposition of behavior into temporally extended sub-tasks is anticipated by the options framework (Sutton et al., 1999) and feudal reinforcement learning (Dayan & Hinton, 1993). More recent hierarchical RL methods such as HIRO (Nachum et al., 2018) learn to set subgoals for lower-level controllers; HGI differs in that the goal inducer segments *observed* behavior rather than proposing goals for a subordinate policy, and the scratchpad is populated externally rather than by a learned manager.

The action model's conditioning on a goal representation connects to universal value function approximators (Schaul et al., 2015) and hindsight experience replay (Andrychowicz et al., 2017), both of which generalize over goal spaces. The scratchpad itself—an explicit, writable region that conditions the output distribution—is related to scratchpad mechanisms for intermediate computation in transformers (Nye et al., 2021) and chain-of-thought prompting (Wei et al., 2022), though in HGI the scratchpad encodes a goal rather than a reasoning trace.

The hierarchical goal inducer performs a form of plan recognition from observations. Classical approaches cast this as planning (Ramirez & Geffner, 2009) or probabilistic inference (Sukthankar et al., 2014); our approach instead uses weak supervision from accessibility-tree event logs annotated by a cloud LLM, a strategy enabled by recent work on LLM-based agents operating in real OS environments (Xie et al., 2024).

The goal-conditioned preference model follows the paradigm of learning reward models from human feedback (Christiano et al., 2017), itself descended from inverse reinforcement learning (Ng & Russell, 2000). The iterative distill–search–distill loop is structurally related to iterated distillation and amplification (Christiano et al., 2018), and the distillation of cloud LLM annotations to a local model is an instance of knowledge distillation (Hinton et al., 2015).

References

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., & Zaremba, W. (2017). Hindsight Experience Replay. *Advances in Neural Information Processing Systems*, 30.

Christiano, P. F., Leike, J., Brown, T., Marber, M., Legg, S., & Amodei, D. (2017). Deep Reinforcement Learning from Human Preferences. *Advances in Neural Information Processing Systems*, 30.

Christiano, P. F., Shlegeris, B., & Amodei, D. (2018). Supervising Strong Learners by Amplifying Weak Experts. *arXiv:1810.08575*.

Dayan, P. & Hinton, G. E. (1993). Feudal Reinforcement Learning. *Advances in Neural Information Processing Systems*, 5.

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. *arXiv:1503.02531*.

Nachum, O., Gu, S., Lee, H., & Levine, S. (2018). Data-Efficient Hierarchical Reinforcement Learning. *Advances in Neural Information Processing Systems*, 31.

Ng, A. Y. & Russell, S. J. (2000). Algorithms for Inverse Reinforcement Learning. *Proceedings of the 17th International Conference on Machine Learning*, 663–670.

Nye, M., Andreassen, A. J., Gur-Ari, G., Michalewski, H., Austin, J., et al. (2021). Show Your Work: Scratchpads for Intermediate Computation with Language Models. *arXiv:2112.00114*.

Ramirez, M. & Geffner, H. (2009). Plan Recognition as Planning. *Proceedings of the 21st International Joint Conference on Artificial Intelligence*, 1778–1783.

Schaul, T., Horgan, D., Gregor, K., & Silver, D. (2015). Universal Value Function Approximators. *Proceedings of the 32nd International Conference on Machine Learning*.

Sukthankar, G., Geib, C., Bui, H. H., Pynadath, D., & Goldman, R. P. (2014). *Plan, Activity, and Intent Recognition: Theory and Practice*. Morgan Kaufmann.

Sutton, R. S., Precup, D., & Singh, S. (1999). Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. *Artificial Intelligence*, 112(1–2), 181–211.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35.

Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., et al. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. *Advances in Neural Information Processing Systems*, 37.